



VerifyPay

KYC Fintech Mobile Application

Complete Technical Documentation

Integration Reference · Developer Onboarding · API Contracts

VERSION	1.0.0
PLATFORM	React Native / Expo 54
LANGUAGE	TypeScript
ARCHITECTURE	Expo Router (File-based)
DATE	July 2026
STATUS	Integration-Ready

This document is intended for:

- OCR & OpenCV Integration Team
- Aadhaar & PAN Verification Team
- Backend API Developers
- QA & Testing Engineers

Table of Contents

1.	Project Overview	3
2.	Quick Start — Clone & Run Locally	4
3.	Adding Real Camera (Selfie) Locally	5
4.	Project Architecture	6
5.	Directory Structure	7
6.	Screen Reference (All 5 Screens)	8
7.	Component Library	12
8.	State Management — AppContext	13
9.	Type Definitions	14
10.	Theme & Design System	15
11.	Mock API Layer	16
12.	Navigation & Routing	17
13.	OCR & OpenCV Integration Guide	18
14.	Aadhaar & PAN Verification Integration	20
15.	Backend API Contracts	22
16.	Environment Variables & Config	24
17.	Testing Guide	25
18.	Known Limitations & TODO	26

1. Project Overview

What VerifyPay is, what it does, and how it is structured

What is VerifyPay?

VerifyPay is a production-ready mobile KYC (Know Your Customer) application built with Expo and React Native. It guides a user through a 5-screen linear flow to collect, verify, and confirm their identity details — including Aadhaar number, PAN card, and a live selfie. The app is currently in integration-ready state: all screens and UI are complete, all state management is wired, and all API call sites are stubbed with clearly-documented mock functions ready to be replaced with real backend calls.

Technology Stack

Layer	Technology	Version	Purpose
Framework	Expo	54.0.27	Cross-platform React Native runtime
Language	TypeScript	5.9.x	Type-safe development
Navigation	Expo Router	6.0.x	File-based routing (like Next.js)
UI	React Native	0.81.5	Native mobile components
Icons	@expo/vector-icons (Ionicons)	15.x	Icon font library
Fonts	Inter (Google Fonts)	0.4.x	UI typography
State	React Context + AsyncStorage	built-in	App-wide state & persistence
Gestures	react-native-gesture-handler	2.28.x	Touch interactions
Animation	React Native Animated API	built-in	Spring/timing animations
Haptics	expo-haptics	15.x	Tactile feedback on interactions
Camera	expo-image-picker (installed)	17.x	Selfie capture (stubbed—see §3)
HTTP	services/mockApi.ts (stub)	—	Replace with real Axios/fetch calls

App Flow

The app follows a strict linear user journey:

Screen	File	Purpose
Landing Screen	index.tsx	Hero + CTA. If signed in !' shows user dashboard with verification status.
Auth Screen	auth.tsx	Sign Up creates account. Sign In validates against stored username. Sign Up must happen first.
Identity Screen	identity.tsx	Collects 12-digit Aadhaar + 10-char PAN. Auto-formats Aadhaar. Validates regex before submission.
Processing Screen	processing.tsx	Animated 4-step verification timeline. Simulates server processing. Auto-advances to Selfie after completion.
Selfie Screen	selfie.tsx	Camera viewfinder UI. Capture !' preview !' confirm flow. On submit, marks selfie as done and navigates to success.

2. Quick Start — Clone & Run Locally

Step-by-step setup for any developer who is new to this project

Prerequisites

Tool	Version	Install
Node.js	18 or 20 LTS	https://nodejs.org
pnpm	9.x	<code>npm install -g pnpm</code>
Expo CLI	via pnpm	<code>pnpm add -g @expo/cli</code>
Expo Go (phone)	Latest	iOS App Store / Google Play
Git	Any	Pre-installed on most systems

Step 1 — Clone the Repository

```
terminal
# Clone the monorepo
git clone <YOUR_REPO_URL> verifypay
cd verifypay
```

Step 2 — Install Dependencies

```
terminal
# Install all workspace packages
pnpm install
```

Step 3 — Run the Expo Dev Server

```
terminal
# Start the Expo development server for the KYC app
pnpm --filter @workspace/fintech-kyc run dev

# OR from the artifact directory directly:
cd artifacts/fintech-kyc
npx expo start
```

Step 4 — Open on Device

- Scan the QR code in Expo Go (Android/iOS)
- Press "w" to open in browser (web preview)
- Press "a" for Android emulator, "i" for iOS Simulator

Monorepo Structure Note

This project lives inside a pnpm workspace monorepo. The KYC app is at `artifacts/fintech-kyc/`. All workspace packages share the root `node_modules`. Always run commands from the root using `pnpm --filter @workspace/fintech-kyc` or from inside `artifacts/fintech-kyc/` directly.

Step 5 — Verify It Works

You should see the VerifyPay landing screen with a dark navy background, the VP logo badge, hero illustration, "Get Started" button, and trust badges (256-bit SSL, RBI Compliant, Instant KYC).

Environment Variables

The dev script automatically reads REPLIT_DEV_DOMAIN, REPLIT_EXPO_DEV_DOMAIN, REPL_ID, and PORT from the Replit environment. For local development outside Replit, start Expo normally:

```
terminal (local, outside Replit)
cd artifacts/fintech-kyc
npx expo start --port 8081
```

3. Adding Real Camera (Selfie) Locally

Replace the mock selfie UI with expo-camera or expo-image-picker

Current Status

The selfie screen currently simulates a camera with an animated oval viewfinder and a mock "Capture" button. expo-image-picker is already listed in package.json and installed. Two approaches are documented below.

Approach A — expo-image-picker (Simplest, Recommended)

expo-image-picker opens the native camera app, lets the user take a photo, and returns a URI. It works on Android, iOS, and has partial web support. No extra native setup needed with Expo Go.

Step 1: Update app.json — Add Camera Permission

```
app.json
{
  "expo": {
    "plugins": [
      "expo-router",
      "expo-font",
      "expo-web-browser",
      [
        "expo-image-picker",
        {
          "cameraPermission": "Allow VerifyPay to use your camera for KYC verification.",
          "microphonePermission": false
        }
      ]
    ]
  }
}
```

Step 2: Replace selfie.tsx — handleCapture function

Find the handleCapture function in app/selfie.tsx and replace it with this implementation:

```
app/selfie.tsx — handleCapture
import * as ImagePicker from 'expo-image-picker';

const [photoUri, setPhotoUri] = useState<string | null>(null);

const handleCapture = async () => {
  // Request camera permission
  const { status } = await ImagePicker.requestCameraPermissionsAsync();
  if (status !== 'granted') {
    Alert.alert('Permission denied', 'Camera access is required for KYC.');
```



```
// Pass base64 to backend: asset.base64
```


Step 3: Show real photo in preview

```
app/selfie.tsx - preview section
// In the preview content section, replace the avatarCircle with:
{photoUri && (
  <Image
    source={{ uri: photoUri }}
    style={{ width: 160, height: 160, borderRadius: 80 }}
    resizeMode="cover"
  />
)}
```

Step 4: Send photo to backend on submit

```
app/selfie.tsx - handleSubmit with real API
const handleSubmit = async () => {
  setStage("submitting");
  const formData = new FormData();
  formData.append("selfie", {
    uri: photoUri,
    name: "selfie.jpg",
    type: "image/jpeg",
  } as any);
  formData.append("userId", user?.username ?? "");

  const res = await fetch("https://YOUR_API/api/selfie", {
    method: "POST",
    body: formData,
    headers: { "Content-Type": "multipart/form-data" },
  });
  const data = await res.json();
  if (data.success) {
    await updateUser({ selfieSubmitted: true });
    setStage("success");
  }
};
```

Approach B — expo-camera (In-App Viewfinder)

For a fully custom in-app camera (no leaving the app), install expo-camera. This gives you full control of the viewfinder, overlays, and real-time processing hooks useful for OCR teams.

```
terminal
npx expo install expo-camera
```

```
expo-camera approach
import { CameraView, useCameraPermissions } from 'expo-camera';
const cameraRef = useRef(null);

const snap = async () => {
  if (cameraRef.current) {
    const photo = await cameraRef.current.takePictureAsync({
      quality: 0.85, base64: true, skipProcessing: false
    });
    // photo.uri, photo.base64 available
    setPhotoUri(photo.uri);
    setStage("preview");
  }
}
```



```
// JSX:
```

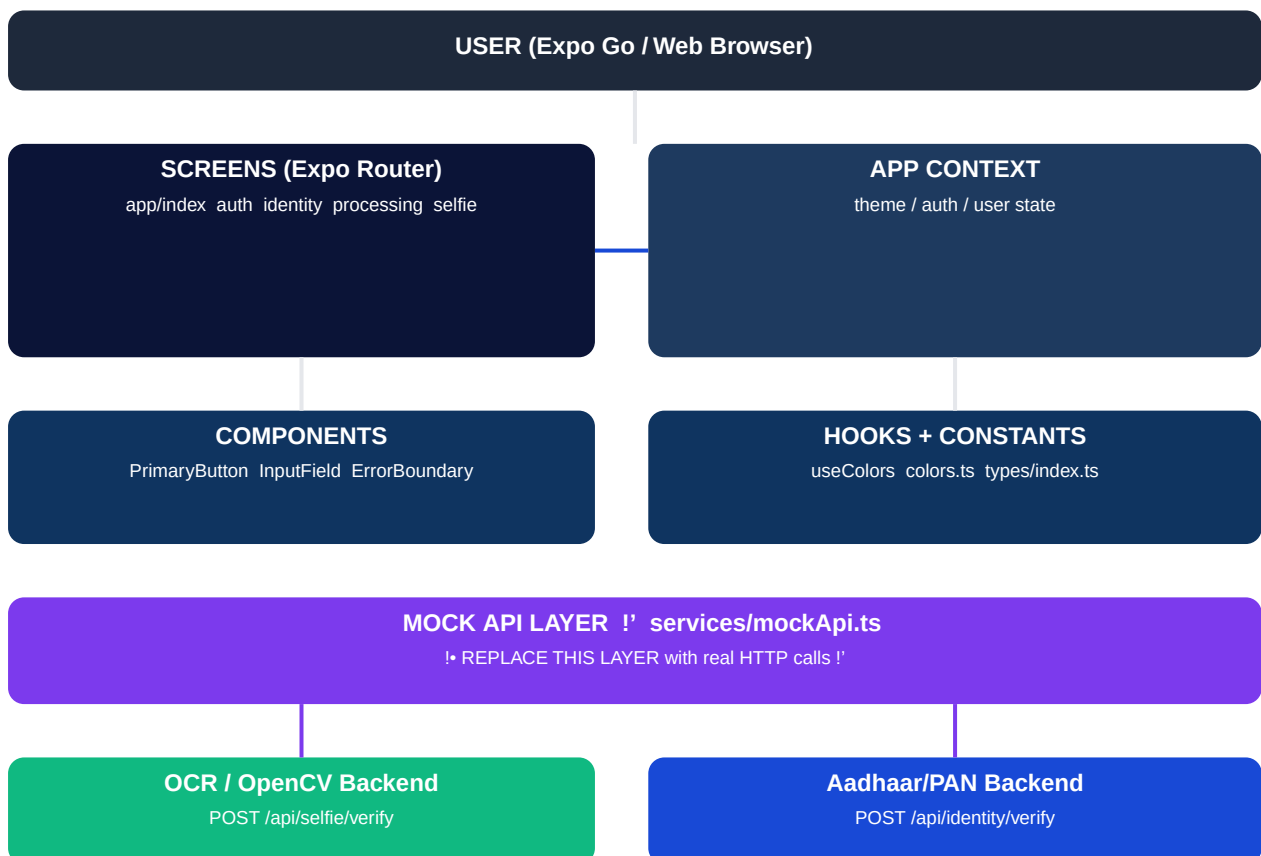

4. Project Architecture

High-level design decisions and data flow

Architecture Philosophy

VerifyPay uses a flat, file-based routing architecture with a single shared context. There is intentionally no Redux, no Zustand, and no complex middleware — the KYC flow is linear and the state is minimal. This makes it easy for integration teams to replace the mock API layer with real HTTP calls without touching the UI layer.

Data Flow Diagram



Key Design Decision

The mock API layer in `services/mockApi.ts` is the **ONLY** file that needs to change to connect to a real backend. Screen components only call `mockSignIn()`, `mockSignUp()`, and `postIdentity()` — they do not make direct `fetch()` calls. This isolation means the integration team can swap the implementation without touching any screen or component file.

5. Directory Structure

Every file and folder explained

Full File Tree

```
directory tree
artifacts/fintech-kyc/
% % % app/                                !• All screens (Expo Router file-based routes)
%   % % % _layout.tsx                    !• Root layout: fonts, providers, stack navigator
%   % % % index.tsx                      !• Screen 1: Landing / Dashboard
%   % % % auth.tsx                       !• Screen 2: Sign Up / Sign In
%   % % % identity.tsx                   !• Screen 3: Aadhaar + PAN entry
%   % % % processing.tsx                 !• Screen 4: Animated verification steps
%   % % % selfie.tsx                     !• Screen 5: Selfie capture + success
%   % % % +not-found.tsx                 !• 404 fallback screen
% % % assets/
%   % % % images/
%       % % % icon.png                   !• App icon (blue shield/VP logo)
%       % % % hero-illustration.png     !• Landing page hero image
% % % components/
%   % % % PrimaryButton.tsx             !• Spring-animated button (solid/outline/ghost)
%   % % % InputField.tsx                 !• Label + border focus + password toggle
%   % % % ErrorBoundary.tsx             !• Class component: catches render errors
%   % % % ErrorFallback.tsx             !• Error UI shown by ErrorBoundary
%   % % % ThemedRoot.tsx                !• Root View with theme background color
%   % % % KeyboardAwareScrollViewCompat.tsx !• Cross-platform scroll wrapper
% % % constants/
%   % % % colors.ts                      !• Full light + dark color palette + radius
% % % contexts/
%   % % % AppContext.tsx                 !• Single app-wide context (theme + auth + user)
% % % hooks/
%   % % % useColors.ts                   !• Returns current theme palette via useApp()
% % % services/
%   % % % mockApi.ts                     !• ALL API calls live here (replace for production)
% % % types/
%   % % % index.ts                       !• TypeScript interfaces for API payloads
% % % app.json                           !• Expo configuration (name, plugins, version)
% % % babel.config.js                     !• Babel preset for Expo
% % % metro.config.js                     !• Metro bundler config
% % % package.json                       !• Dependencies (workspace package)
% % % tsconfig.json                      !• TypeScript config (@/* path alias)
```

Path Alias

The @/* alias is configured in tsconfig.json and resolves to the project root:

```
tsconfig.json + usage
// tsconfig.json (relevant section)
{
  "compilerOptions": {
    "paths": {
      "@/*": ["./*"]
    }
  }
}
```



```
// Usage in any file:
```

```
import { useColors } from '@hooks/useColors'; // '
```

```
import colors from '@constants/colors'; // '
```

```
import { useApp } from '@contexts/AppContext'; // '
```

6. Screen Reference — All 5 Screens

Detailed breakdown of each screen: purpose, state, navigation, integration points

Screen 1 — Landing / Dashboard (app/index.tsx)

Property	Value
Route	/ (root)
File	app/index.tsx
Navigation	Entry point. No back button. Links to /auth and /identity.
Auth-gated	Yes — renders two different UIs based on isSignedIn

Landing Mode (not signed in)

- VP logo badge + theme toggle (sun/moon) in top bar
- hero-illustration.png centred in screen
- "Verify your identity in minutes" headline + subtitle
- Trust badges: 256-bit SSL, RBI Compliant, Instant KYC
- "Get Started" !' navigates to /auth
- "Already have an account? Sign in" !' /auth
- Fade + slide-up entrance animation

Dashboard Mode (signed in)

- Greeting ("Welcome back, [username]") + sign-out button
- Theme toggle button
- Blue avatar card with initials, KYC status badge, member since / account type / status stats
- IDENTITY DETAILS card: masked Aadhaar (XXXX XXXX 1234), masked PAN, selfie status
- VERIFICATION STATUS timeline: 3 steps with done/pending dots
- Contextual CTA: "Complete Verification" / "Submit Selfie" / "KYC fully complete" banner

Integration Points

- updateUser({ aadhaar, pan, verifiedAt, selfieSubmitted }) — called from identity.tsx and selfie.tsx
- signOut() — clears isSignedIn from AsyncStorage, returns to landing mode
- No direct API calls in this screen

Screen 2 — Auth (app/auth.tsx)

Property	Value
Route	/auth
File	app/auth.tsx
Navigation	Back arrow !' /. On Sign Up success !' /identity. On Sign In success !' /.
Auth-gated	Partially — Sign In tab is disabled until Sign Up is complete

Behaviour

- Toggle between Sign Up and Sign In tabs
- Sign In tab shows a lock notice and is disabled if isSignedUp is false
- Sign Up: validates username (non-empty) + password ("e6 chars), calls mockSignUp(), then calls AppContext.signUp({ username }), navigates to /identity
- Sign In: validates same fields, calls mockSignIn(), then calls AppContext.signIn(username) which checks username matches the registered account
- Global error box shown for server-level errors
- Haptic feedback on success (NotificationFeedbackType.Success) and error (.Error)

Integration Points — Replace mockSignUp / mockSignIn

```
services/mockApi.ts
// services/mockApi.ts – replace these functions:
```



```
export async function mockSignUp(credentials: AuthCredentials) {
```



```
const res = await fetch("https://YOUR_API/api/auth/register", {
```



```
export async function mockSignIn(credentials: AuthCredentials) {
```

```
const res = await fetch("https://YOUR_API/api/auth/login", {
```


Screen 3 — Identity Verification (app/identity.tsx)

Property	Value
Route	/identity
File	app/identity.tsx
Navigation	Back !' /auth. On success !' /processing.
Auth-gated	No explicit guard, but reached after sign up

Fields & Validation

Field	Format	Validation	Auto-format
Aadhaar Number	XXXX XXXX XXXX	12 digits, numeric only	Yes — spaces inserted at 4/8
PAN Number	ABCDE1234F	5 letters + 4 digits + 1 letter	Auto uppercase

Behaviour

- Progress bar shown at 66% (step 2 of 3)
- Both fields validated client-side before API call
- On success: navigates to /processing. Does NOT yet call updateUser — the Processing screen sets verifiedAt
- Error messages shown inline under each field
- Haptic feedback on error and success

Integration Points — Replace postIdentity

```
services/mockApi.ts
// services/mockApi.ts — replace postIdentity:
export async function postIdentity(payload: IdentityPayload): Promise<IdentityResponse> {
  const res = await fetch("https://YOUR_API/api/identity/verify", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": "Bearer " + userToken, // add your auth token
    },
    body: JSON.stringify({
      aadhaar_number: payload.aadhaarNumber.replace(/\s/g, ""),
      pan_number: payload.panNumber,
    }),
  });
  const data = await res.json();
  return {
    success: data.status === "verified",
    message: data.message,
    verificationId: data.verification_id,
  };
}
```

Screen 4 — Processing (app/processing.tsx)

Property	Value
Route	/processing
File	app/processing.tsx
Navigation	No back button. Auto-navigates to /selfie after ~8.4 seconds.
Auth-gated	No

Animation Sequence

- Step 1 (1.6s): "Connecting to servers" !' active
- Step 2 (2.2s): "Verifying credentials" !' active
- Step 3 (2.6s): "Checking identity records" !' active
- Step 4 (2.0s): "Fetching government data" !' active
- All done: calls updateUser({ verifiedAt: new Date().toISOString() }), shows success badge, waits 1s, pushes / selfie

Integration Points

This screen currently auto-advances after a fixed timer. For production, replace the timer-based logic with a polling mechanism or WebSocket subscription to your backend verification job:

```
processing.tsx – polling approach
// Poll backend for verification status (replace setTimeout logic):
const pollVerification = async (verificationId: string) => {
  const maxAttempts = 20;
  for (let i = 0; i < maxAttempts; i++) {
    const res = await fetch(`https://YOUR_API/api/verification/${verificationId}`);
    const data = await res.json();
    if (data.status === "verified") {
      await updateUser({ verifiedAt: data.verified_at });
      router.push("/selfie");
      return;
    }
    await new Promise(r => setTimeout(r, 1500));
  }
  // Handle timeout / failure
};
```

Screen 5 — Selfie (app/selfie.tsx)

Property	Value
Route	/selfie
File	app/selfie.tsx
Navigation	Back !' /processing. On success !' shows inline SuccessView, then /
Auth-gated	No

States (Stage Machine)

Stage	What user sees	Transition
idle	Animated oval viewfinder, scan line, "Capture Selfie" button	!' preview (on capture)
preview	Face icon or real photo, "Retake" + "Submit" buttons	!' idle (retake), submitting (submit)
submitting	Loading spinner in Submit button	!' success
success	Full-screen success view: checkmark, 3 completion items, "View Dashboard" button	!' / (on button press)

Integration Points

- Stage idle !' preview: replace with real camera call (see §3)
- Stage preview !' submitting: replace handleSubmit with multipart/form-data POST to /api/selfie
- On success: calls updateUser({ selfieSubmitted: true }) — this updates the dashboard
- The SuccessView component is defined inline in the same file — move to components/ if reusing

7. Component Library

Reusable UI components, their props, and usage

PrimaryButton (components/PrimaryButton.tsx)

A spring-animated touchable button with three visual variants.

```
PrimaryButton – Props
interface PrimaryButtonProps {
  label: string;           // Button text
  onPress: () => void;      // Tap handler
  loading?: boolean;       // Shows ActivityIndicator, disables tap
  disabled?: boolean;      // Greys out button
  variant?: "solid" | "outline" | "ghost"; // default: "solid"
  style?: ViewStyle;       // Additional container style
}
```

Usage examples:

```
Usage
// Solid (default) – used on all primary CTAs
<PrimaryButton label="Get Started" onPress={() => router.push("/auth")} />

// Outline – used for secondary actions (Retake on selfie screen)
<PrimaryButton label="Retake" onPress={handleRetake} variant="outline" />

// Loading state
<PrimaryButton label="Submitting..." onPress={() => {}} loading />
```

- Used in: index.tsx, auth.tsx, identity.tsx, processing.tsx, selfie.tsx
- Colours sourced from useColors() — automatically adapts to light/dark theme
- Spring animation: scale 0.97 on press-in, spring back on press-out

InputField (components/InputField.tsx)

Labelled text input with focus border, inline error/hint, and password toggle.

```
InputField – Props
interface InputFieldProps extends TextInputProps {
  label: string;           // Label shown above the input
  error?: string;          // Red error message shown below
  hint?: string;           // Grey hint shown below (if no error)
  isPassword?: boolean;    // Shows eye-toggle icon
  // + all standard TextInput props (value, onChangeText, etc.)
}
```

- Used in: auth.tsx (username, password), identity.tsx (aadhaar, pan)
- Forwards ref — supports onSubmitEditing chaining (username ! password)
- Border colour changes to primary blue on focus, red on error

ErrorBoundary (components/ErrorBoundary.tsx)

Class component that catches any uncaught render error and renders ErrorFallback instead of crashing the app. Wraps the entire navigator in _layout.tsx.

- Props: FallbackComponent (default: ErrorFallback), onError?: (error, info) => void
- Used in: _layout.tsx (root wrap)

ThemedRoot (components/ThemedRoot.tsx)

A simple View that reads the current theme from AppContext and applies the correct background colour.
Used internally by `_layout.tsx`.

KeyboardAwareScrollViewCompat (components/ KeyboardAwareScrollViewCompat.tsx)

Platform-aware scroll wrapper. Uses `react-native-keyboard-controller` on native for smooth keyboard avoidance, falls back to standard `ScrollView` on web.

8. State Management — AppContext

Single source of truth: theme, auth, and user data

AppContext Overview

AppContext (contexts/AppContext.tsx) is the only global state store. It wraps the entire app in `_layout.tsx` via `<AppProvider>`. All screens and hooks read from it via the `useApp()` hook.

Context Interface

```
contexts/AppContext.tsx
interface AppState {
  // Theme
  theme: "light" | "dark";
  toggleTheme: () => void; // Toggles & persists to AsyncStorage

  // Auth flags
  isSignedUp: boolean; // An account has been created this session
  isSignedIn: boolean; // User is currently logged in

  // User data
  user: UserInfo | null;

  // Actions
  signUp: (info: UserInfo) => Promise<void>;
  signIn: (username: string) => Promise<boolean>; // returns false if username mismatch
  signOut: () => Promise<void>;
  updateUser: (info: Partial<UserInfo>) => Promise<void>;
}

interface UserInfo {
  username: string;
  aadhaar?: string; // raw 12-digit string (no spaces)
  pan?: string; // uppercase 10-char string
  verifiedAt?: string; // ISO date string set by processing.tsx
  selfieSubmitted?: boolean; // set by selfie.tsx
}
```

Persistence

Key	AsyncStorage Key	What is stored	Cleared on
theme	"theme"	"light" or "dark"	Never (user preference)
user	"user"	Full UserInfo JSON object	Never (re-loaded on app start)
isSignedIn	"isSignedIn"	"true" string	signOut() call

How screens use AppContext

```
Usage pattern
// Any screen or component – just import useApp:
```

```
import { useApp } from '@contexts/AppContext';
```


useColors Hook

```
hooks/useColors.ts
// hooks/useColors.ts
import { useApp } from '@contexts/AppContext';
import colors from '@constants/colors';

export function useColors() {
  const { theme } = useApp();
  const palette = theme === "dark" ? colors.dark : colors.light;
  return { ...palette, radius: colors.radius };
}

// Usage in any component:
const colors = useColors();
<View style={{ backgroundColor: colors.background }} />
```

9. Type Definitions (types/index.ts)

All TypeScript interfaces used across the project

```
types/index.ts – complete
// types/index.ts – complete file

// Used by mockSignIn and mockSignUp
export interface AuthCredentials {
  username: string;
  password: string;
}

// Sent to POST /api/identity/verify
export interface IdentityPayload {
  aadhaarNumber: string; // 12 digits, spaces allowed (stripped before send)
  panNumber: string;      // ABCDE1234F format
}

// Returned from POST /api/identity/verify
export interface IdentityResponse {
  success: boolean;
  message: string;
  verificationId?: string; // e.g. "VRF-ABC123" – reference for polling
}

// Used in processing.tsx step animation
export type VerificationStep = {
  id: string;
  label: string;
  status: "pending" | "active" | "done";
};
```

For Integration Teams

When connecting a real backend, you may need to extend IdentityResponse to include server-returned fields like confidence_score, name_match, dob, or address. Add new fields to the interface in types/index.ts and update the postIdentity function in services/mockApi.ts to map the server response to this shape.

Adding New Types

All types live in types/index.ts. When adding selfie or OCR response types, add them here:

```
types/index.ts – selfie types to add
// types/index.ts – add for selfie integration
export interface SelfiePayload {
  selfieUri: string; // local file URI
  selfieBase64?: string; // optional base64 for direct upload
  userId: string;
}

export interface SelfieResponse {
  success: boolean;
  message: string;
  livenessScore?: number; // 0-1 liveness detection score
  faceMatchScore?: number; // 0-1 match against ID document
}
```


10. Theme & Design System (constants/colors.ts)

Full colour palette, both light and dark

Light Theme

Token	Hex Value	Used For
background	#FFFFFF	Screen background
card	#F7F8FF	Card surfaces, input backgrounds
primary	#1849D6	CTA buttons, active borders, icons
primaryLight	#EEF2FF	Icon backgrounds, info boxes
text	#0A0A14	Primary body text
mutedForeground	#6B7280	Labels, hints, placeholders
border	#E5E7EB	Card borders, dividers
destructive	#EF4444	Error messages, error borders
success	#10B981	Verified badges, checkmarks
successLight	#ECFDF5	Success banner background
muted	#F3F4F6	Toggle background, inactive elements

Dark Theme

Token	Hex Value	Used For
background	#080B14	Screen background (deep navy)
card	#0F1322	Card surfaces
primary	#4A7DFF	CTA buttons, active borders, icons
primaryLight	#111D3D	Icon backgrounds, info boxes
text	#E8EEFF	Primary body text
mutedForeground	#6B7FA8	Labels, hints, placeholders
border	#1A2340	Card borders, dividers
destructive	#F87171	Error messages
success	#34D399	Verified badges
successLight	#022C22	Success banner background
muted	#111827	Toggle background

Shared Values

```
constants/colors.ts
colors.radius = 14 // Base border radius for cards and buttons
```

Typography

Font	Weight	Used For

Inter_400Regular

Inter_500Medium

Inter_600SemiBold

Inter_700Bold

Screen titles, headlines, card names

11. Mock API Layer (services/mockApi.ts)

The integration team — replace these 3 functions for production

Integration Team Instruction

This is the ONLY file you need to modify to connect the app to a real backend. Do not modify any screen file or component. Simply replace the function bodies with real fetch() calls that return the same response shape.

Function 1 — mockSignUp

```
mockSignUp
// Current implementation (MOCK):
export async function mockSignUp(credentials: AuthCredentials)
  : Promise<{ success: boolean; message: string }> {
  await delay(1000); // simulates network latency
  // Basic validation only – returns success if username + 6+ char password
  return { success: true, message: "Account created successfully." };
}

// Required return shape:
{ success: boolean, message: string }
```

Function 2 — mockSignIn

```
mockSignIn
// Current implementation (MOCK):
export async function mockSignIn(credentials: AuthCredentials)
  : Promise<{ success: boolean; message: string }> {
  await delay(900);
  // Only checks format – actual username match is done in AppContext.signIn()
  return { success: true, message: "Signed in successfully." };
}

// Required return shape:
{ success: boolean, message: string }
```

Function 3 — postIdentity

```
postIdentity
// Current implementation (MOCK):
export async function postIdentity(payload: IdentityPayload)
  : Promise<IdentityResponse> {
  await delay(800);
  const aadhaarValid = /^d{12}$/.test(payload.aadhaarNumber.replace(/\s/g, ""));
  const panValid = /^[A-Z]{5}[0-9]{4}[A-Z]{1}$/.test(payload.panNumber);
  if (!aadhaarValid) return { success: false, message: "Invalid Aadhaar number." };
  if (!panValid) return { success: false, message: "Invalid PAN number." };
  return {
    success: true, message: "Identity submitted.",
    verificationId: "VRF-" + Date.now().toString(36).toUpperCase(),
  };
}
```

```
// Required return shape:
```


12. Navigation & Routing

Expo Router file-based navigation — all routes and transitions

Route Map

Route	File	Transition	Back?
/	app/index.tsx	Root — no transition	No
/auth	app/auth.tsx	slide_from_right	Yes (!' /)
/identity	app/identity.tsx	slide_from_right	Yes (!' /auth)
/processing	app/processing.tsx	slide_from_right	No back button shown
/selfie	app/selfie.tsx	slide_from_right	Yes (!' /processing)

Stack Configuration (_layout.tsx)

```
app/_layout.tsx
<Stack screenOptions={{ headerShown: false, animation: "slide_from_right" }}>
  <Stack.Screen name="index" />
  <Stack.Screen name="auth" />
  <Stack.Screen name="identity" />
  <Stack.Screen name="processing" />
  <Stack.Screen name="selfie" />
</Stack>
```

Navigation Calls Used in Screens

Screen	Call	Target
index.tsx	router.push("/auth")	/auth
auth.tsx	router.replace("/identity")	/identity (on sign up)
auth.tsx	router.replace("/")	/ (on sign in)
auth.tsx	router.back()	Previous screen
identity.tsx	router.push("/processing")	/processing
processing.tsx	router.push("/selfie")	/selfie (auto after 8.4s)
selfie.tsx	router.replace("/")	/ (from success view)

router.replace vs router.push

router.replace() replaces the current route in the stack (no back button to that screen). This is intentional on auth!identity and auth!/ so the user cannot press back into the sign-up form after completing it. router.push() adds to the stack and allows back navigation.

Deep Linking

The app scheme is fintech-kyc (set in app.json). Deep links follow the pattern:

```
Deep link scheme
fintech-kyc://          !' / (Landing / Dashboard)
fintech-kyc://auth      !' /auth
```


13. OCR & OpenCV Integration Guide

For the team adding document scanning, face detection, and liveness checks

Scope of This Section

This section is specifically for the team integrating OCR (Optical Character Recognition) and OpenCV-based image processing. It covers where to hook into the app, what data is available, what the backend must return, and recommended libraries.

What OCR Needs to Process

Document	Data to Extract	Source Screen
Aadhaar Card (front)	Aadhaar number (12 digits), Name, DOB, Address, Gender	identity.tsx (add camera scan)
PAN Card (front)	PAN number, Name, DOB, Father name	identity.tsx (add camera scan)
Live Selfie	Face detection, Liveness check, Face-to-ID matching	selfie.tsx

Where to Hook In — identity.tsx (OCR for Aadhaar & PAN)

Currently the user types their Aadhaar and PAN manually. OCR integration adds a "Scan Card" button that opens the camera, captures the document, and auto-fills the fields. The hook point is in identity.tsx:

```
identity.tsx — OCR hook
// identity.tsx — add alongside manual entry fields:

import * as ImagePicker from 'expo-image-picker';

const handleScanAadhaar = async () => {
  const result = await ImagePicker.launchCameraAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    quality: 1.0,      // High quality for OCR accuracy
    base64: true,
  });
  if (!result.canceled) {
    const base64 = result.assets[0].base64;
    // Send to OCR backend:
    const ocr = await postOcrScan({ imageBase64: base64, docType: "aadhaar" });
    if (ocr.success) {
      setAadhaar(formatAadhaar(ocr.aadhaarNumber)); // auto-fill
    }
  }
};
```

New API Function — postOcrScan (add to services/mockApi.ts)

services/mockApi.ts — OCR function

```
// Add to services/mockApi.ts:
```



```
export interface OcrPayload {
```



```
export interface OcrResponse {
```



```
export async function postOcrScan(payload: OcrPayload): Promise<OcrResponse> {
```

```
const res = await fetch("https://YOUR_OCR_API/api/ocr/scan", {
```


Where to Hook In — selfie.tsx (Face Detection + Liveness)

```
selfie.tsx - handleSubmit with OCR/liveness
// selfie.tsx - replace handleSubmit:

const handleSubmit = async () => {
  setStage("submitting");

  // 1. Upload selfie image to OCR/OpenCV backend
  const formData = new FormData();
  formData.append("selfie", { uri: photoUri, name: "selfie.jpg", type: "image/jpeg" } as any);
  formData.append("userId", user!.username);

  const res = await fetch("https://YOUR_API/api/selfie/verify", {
    method: "POST",
    body: formData,
  });
  const data: SelfieResponse = await res.json();

  if (!data.success) {
    // Show error: data.message (e.g. "Liveness check failed")
    setStage("idle");
    return;
  }

  // 2. Store result and mark complete
  await updateUser({ selfieSubmitted: true });
  setStage("success");
};
```

Recommended OCR Libraries (Backend Side)

Library	Language	Use Case
Tesseract.js / Tesseract (pytesseract)	JS / Python	General OCR text extraction from document images
OpenCV (cv2)	Python	Image preprocessing, deskew, noise removal, face detection
AWS Textract	Any (REST)	Production-grade document OCR with confidence scores
Google Vision API	Any (REST)	OCR + face detection in one API
DeepFace	Python	Face recognition and liveness detection
face-api.js	JavaScript/Node	In-app face detection (runs on device with TFLite)

Image Preprocessing Recommendations

- Send images at minimum 1200×800px for reliable OCR
- Apply grayscale + adaptive thresholding before OCR to improve accuracy
- Detect and reject images where confidence < 0.7
- For Aadhaar: mask digits 1–8 in stored/logged versions (display as XXXX XXXX 1234)
- For selfie liveness: require blink or head-turn challenge on production

14. Aadhaar & PAN Verification Integration

For the team connecting government identity APIs

Scope of This Section

This section is for the team integrating real Aadhaar OTP-based authentication and PAN verification. It covers the API contract the frontend expects, what government APIs to use, and how to wire everything in.

Current Frontend Validation (Client-Side Only)

The identity.tsx screen validates format locally before calling postIdentity():

```
identity.tsx - client validation
// Aadhaar: must be 12 digits
const aadhaarValid = /\d{12}$/.test(aadhaar.replace(/\s/g, ''));

// PAN: AAAAA9999A format (5 letters + 4 digits + 1 letter)
const panValid = /^[A-Z]{5}[0-9]{4}[A-Z]{1}$/.test(pan.toUpperCase());
```

These are FORMAT checks only. Real verification is done server-side. You must return the correct response shape from your backend.

Backend Contract for POST /api/identity/verify

```
API Contract
// Frontend sends:
POST /api/identity/verify
Content-Type: application/json

{
  "aadhaar_number": "123456789012", // 12 digits, no spaces
  "pan_number": "ABCDE1234F"        // uppercase 10-char
}

// Frontend expects (IdentityResponse shape):
{
  "success": true,
  "message": "Identity verified successfully.",
  "verificationId": "VRF-ABC123XY" // used for polling in processing.tsx
}

// On failure:
{
  "success": false,
  "message": "Aadhaar number not found in UIDAI records."
}
```

Government APIs for Production

API	Provider	Endpoint / SDK	Use
UIDAI Aadhaar OTP	NSDL / AuthBridge / Signzy	Partner API	Verify Aadhaar via OTP sent to linked mobile
Aadhaar XML / eKYC	UIDAI licensed agencies	Certified API	Full KYC with name, address, photo from UIDAI

REST API

Verify PAN against Income Tax
database

--

PAN + Aadhaar Link

ITD API

REST API

Check if PAN is linked to
Aadhaar (mandatory)

Important Compliance Note

Aadhaar-based authentication in India requires UIDAI-licensed Authentication User Agency (AUA) / e-KYC User Agency (KUA) certification. You cannot call UIDAI APIs directly. You must use a licensed intermediary such as NSDL, AuthBridge, Signzy, IDfy, or Karza. Ensure your backend handles biometric/demographic data per UIDAI circular and DPDP Act 2023 guidelines.

Adding OTP Step for Aadhaar (Recommended Flow)

UIDAI authentication sends an OTP to the Aadhaar-linked mobile. Add a new screen or modal for OTP entry after identity.tsx submits:

```
Aadhaar OTP flow
// New flow: identity.tsx !' OTP modal !' processing.tsx

// Step 1: identity.tsx calls initAadhaarOtp (new function)
export async function initAadhaarOtp(aadhaarNumber: string)
  : Promise<{ success: boolean; requestId: string; message: string }> {
  const res = await fetch("https://YOUR_API/api/aadhaar/send-otp", {
    method: "POST",
    body: JSON.stringify({ aadhaar_number: aadhaarNumber }),
    headers: { "Content-Type": "application/json" },
  });
  return res.json();
}

// Step 2: user enters 6-digit OTP in modal
// Step 3: verify OTP
export async function verifyAadhaarOtp(requestId: string, otp: string)
  : Promise<IdentityResponse> {
  const res = await fetch("https://YOUR_API/api/aadhaar/verify-otp", {
    method: "POST",
    body: JSON.stringify({ request_id: requestId, otp }),
    headers: { "Content-Type": "application/json" },
  });
  return res.json();
}
```

updateUser — Fields to Populate After Verification

```
AppContext.updateUser call
// Call after successful identity verification:
await updateUser({
  aadhaar: payload.aadhaarNumber, // stored for dashboard display
  pan: payload.panNumber,
  verifiedAt: new Date().toISOString(), // set by processing.tsx already
});

// The dashboard (index.tsx) will automatically show:
// - Masked Aadhaar: XXXX XXXX 1234
// - Masked PAN: AB*****F
// - "KYC Verified" badge in green
```

15. Backend API Contracts

Complete specification for all endpoints the frontend calls

Overview

The frontend currently calls 3 functions in `services/mockApi.ts`. Each must have a real backend counterpart. All endpoints should be prefixed with your base URL (e.g. `https://api.verifypay.in/v1`).

1. POST `/api/auth/register`

Field	Details
Method	POST
Path	<code>/api/auth/register</code>
Auth	None (public)
Request body	<code>{ "username": "string", "password": "string" }</code>
Success 200	<code>{ "success": true, "message": "Account created.", "token": "JWT..." }</code>
Error 400	<code>{ "success": false, "message": "Username already taken." }</code>
Called from	<code>auth.tsx !' mockSignUp()</code>

2. POST `/api/auth/login`

Field	Details
Method	POST
Path	<code>/api/auth/login</code>
Auth	None (public)
Request body	<code>{ "username": "string", "password": "string" }</code>
Success 200	<code>{ "success": true, "message": "Signed in.", "token": "JWT..." }</code>
Error 401	<code>{ "success": false, "message": "Invalid credentials." }</code>
Called from	<code>auth.tsx !' mockSignIn()</code>

3. POST `/api/identity/verify`

Field	Details
Method	POST
Path	<code>/api/identity/verify</code>
Auth	Bearer JWT token
Request body	<code>{ "aadhaar_number": "123456789012", "pan_number": "ABCDE1234F" }</code>
Success 200	<code>{ "success": true, "message": "...", "verificationId": "VRF-ABC123" }</code>
Error 422	<code>{ "success": false, "message": "Aadhaar not found in UIDAI records." }</code>
Called from	<code>identity.tsx !' postIdentity()</code>

4. POST /api/selfie/verify (to be added)

Field	Details
Method	POST
Path	/api/selfie/verify
Auth	Bearer JWT token
Request body	multipart/form-data — field "selfie" (JPEG), field "userId"
Success 200	{ "success": true, "message": "...", "livenessScore": 0.97, "faceMatchScore": 0.93, "selfieId": "SLF-XYZ" }
Error 422	{ "success": false, "message": "Liveness check failed. Please retry." }
Called from	selfie.tsx !' handleSubmit() (after real camera integration)

5. POST /api/ocr/scan (to be added for OCR team)

Field	Details
Method	POST
Path	/api/ocr/scan
Auth	Bearer JWT token
Request body	{ "imageBase64": "base64string...", "docType": "aadhaar" "pan" }
Success 200	{ "success": true, "aadhaarNumber": "1234...", "name": "...", "confidence": 0.95, "message": "..." }
Error 422	{ "success": false, "confidence": 0.3, "message": "Image too blurry. Please retake." }
Called from	identity.tsx !' handleScanAadhaar() / handleScanPan() (after OCR integration)

6. GET /api/verification/:verificationId (polling)

Field	Details
Method	GET
Path	/api/verification/{verificationId}
Auth	Bearer JWT token
Response (pending)	{ "status": "processing", "progress": 0.6 }
Response (done)	{ "status": "verified", "verified_at": "2026-07-14T10:00:00Z" }
Response (failed)	{ "status": "failed", "message": "Could not verify identity." }
Called from	processing.tsx — replace timer with poll loop

16. Environment Variables & Configuration

All config values, how to set them locally and in production

Current Expo Environment Variables

Variable	Set By	Used In	Description
PORT	Replit system	dev script	Port for Expo dev server
REPLIT_DEV_DOMAIN	Replit system	dev script	Public domain for preview
REPLIT_EXPO_DEV_DOMAIN	Replit system	dev script	Expo-specific tunnel domain
REPL_ID	Replit system	dev script	Replit repl identifier

Adding Backend API URL (for Integration Teams)

Create a .env file in artifacts/fintech-kyc/ (do NOT commit this file):

```
.env
# artifacts/fintech-kyc/.env
EXPO_PUBLIC_API_BASE_URL=https://api.verifypay.in/v1
EXPO_PUBLIC_OCR_API_URL=https://ocr.verifypay.in
```

Then use it in services/mockApi.ts:

```
services/mockApi.ts
const API_BASE = process.env.EXPO_PUBLIC_API_BASE_URL ?? 'http://localhost:3000';

export async function postIdentity(payload: IdentityPayload) {
  const res = await fetch(`${API_BASE}/api/identity/verify`, { ... });
  return res.json();
}
```

EXPO_PUBLIC_ Prefix Required

Expo only exposes environment variables prefixed with EXPO_PUBLIC_ to client-side code. Secret keys (API secret keys, signing keys) must NEVER use this prefix — they should stay server-side only and never be bundled into the app.

17. Testing Guide

How to test each screen and integration point

Manual Testing Checklist

Test	Steps	Expected Result
Landing — dark mode	Open app	Navy background, VP badge, hero image, trust badges
Landing — light toggle	Tap sun icon	White background throughout, persists on restart
Sign Up	Enter username + 6+ char password, tap Create	Navigates to Identity screen
Sign In guard	Tap Sign In tab before Sign Up	Lock notice visible, button disabled
Sign In	After Sign Up, tap Sign In, same credentials	Returns to dashboard
Wrong username login	Different username than registered	"does not match" error shown
Identity — Aadhaar format	Type 12 digits	Auto-formats as XXXX XXXX XXXX
Identity — bad PAN	Enter "ABCD12345"	Inline error shown, no API call
Identity — valid data	ABCDE1234F + 12 digits, submit	Processing screen loads
Processing animation	Observe	4 steps animate in sequence over 8.4 seconds
Selfie capture	Tap "Capture Selfie"	Mock photo shown, Retake + Submit visible
Selfie submit	Tap Submit after capture	Success view with checkmarks
Dashboard — post KYC	Complete full flow	Masked Aadhaar, masked PAN, selfie checkmark, "KYC fully complete" banner
Sign out	Tap log-out icon	Returns to landing screen (not signed in)

Testing Integration Points

To test against a real backend before all endpoints are live, you can use partial integration:

```
Partial integration testing
// services/mockApi.ts — test ONE real endpoint while others remain mocked:

export async function postIdentity(payload: IdentityPayload): Promise<IdentityResponse> {
  if (process.env.EXPO_PUBLIC_USE_REAL_API === "true") {
    const res = await fetch(`${API_BASE}/api/identity/verify`, {
      method: "POST",
      body: JSON.stringify(payload),
      headers: { "Content-Type": "application/json" },
    });
    return res.json();
  }
  // Fall back to mock
  await delay(800);
  return { success: true, message: "Mock verified.", verificationId: "VRF-MOCK" };
}
```

18. Known Limitations & TODO

Current state vs. production-ready state

Current Limitations

Item	Status	What to Do
Real camera (selfie)	STUBBED — mock oval UI	Integrate expo-image-picker (\$3)
Aadhaar OCR scan	NOT PRESENT — manual entry only	Add scan button + OCR API (\$13)
PAN OCR scan	NOT PRESENT — manual entry only	Add scan button + OCR API (\$13)
Auth backend	STUBBED — mockSignIn/mockSignUp	Replace with real auth API (\$15)
Identity verification	STUBBED — regex only	Replace postIdentity (\$14, \$15)
Aadhaar OTP	NOT PRESENT	Add OTP modal after Aadhaar submit (\$14)
JWT token storage	NOT PRESENT — no token handling	Store JWT in SecureStore after login
Session expiry	NOT PRESENT	Add token expiry check on app resume
Processing polling	TIMER-BASED — fixed 8.4s	Replace with real polling (\$6 / \$15)
Network error handling	PARTIAL — only shows message	Add retry logic and offline detection
Accessibility	PARTIAL	Add accessibilityLabel to all interactive elements
Tablet layout	NOT SUPPORTED	supportsTablet: false in app.json
Push notifications	NOT PRESENT	Add expo-notifications for KYC status updates

Security Checklist for Production

- Store JWT/auth tokens in expo-secure-store (NOT AsyncStorage — AsyncStorage is not encrypted)
- Never log or display full Aadhaar number in UI or console
- Mask Aadhaar to show only last 4 digits everywhere in UI (already done in dashboard)
- All API calls must use HTTPS — never HTTP in production
- Implement certificate pinning for API calls to prevent MITM attacks
- Add rate limiting on auth endpoints to prevent brute-force
- Implement biometric lock (expo-local-authentication) for re-entry after background

Suggested Next Steps (Priority Order)

Priority	Task	Owner
P0	Wire real auth API (register + login) in mockApi.ts	Auth team
P0	Wire real Aadhaar + PAN verify API in mockApi.ts	Aadhaar/PAN team
P0	Integrate expo-image-picker for selfie capture	Frontend team
P1	Add OCR scan button to identity screen	OCR team
P1	Add selfie upload to OCR/liveness backend	OCR team
P1	Replace processing timer with real polling	Backend team
P1	Move JWT storage from AsyncStorage to SecureStore	Frontend team
P2	Add Aadhaar OTP verification modal	Aadhaar team

Add error retry logic for network failures

Frontend team

Add push notifications for KYC status updates

Frontend team



VerifyPay KYC

Complete Technical Documentation

Version 1.0.0 · July 2026

This document covers the full VerifyPay KYC mobile application: project structure, all 5 screens, state management, API contracts, OCR integration guide, Aadhaar/PAN verification guide, and developer onboarding from clone to run.

For questions, contact the VerifyPay engineering team.